

# Using a Coding Agent for Independent Research

---

## CSC / CTC 492 — Codex, GitHub, and Managing Your Own Project

### Contents

1. CSC / CTC 492 — Codex, GitHub, and Managing Your Own Project
2. Working with an agent, and understanding the result
3. What does the course expect at the end?
4. What is Codex?
5. What should you give the agent, and what should you keep?
6. Using Codex for the research, not only the code
7. Initial Setup
8. GitHub access: a key and a token
9. The repository
10. Milestones, written before any issue
11. Writing an issue
12. The cycle, repeated for every issue
13. One issue, one branch
14. Reviewing what Codex produced
15. Committing every change, then merging
16. Recording what the agent did
17. The research notebook, in the repository wiki
18. Writing the paper in LaTeX
19. Once a week
20. The final: questions about your own deliverables

### Appendix

- A. What each version-control action does
- B. Paper format: sections and page budget

- C. Running Codex against your own model
- D. Using VS Code with WSL
- E. Glossary
- F. Further reading

## CSC / CTC 492 — Codex, GitHub, and Managing Your Own Project

This guide is about running your independent research project with a *coding agent* doing part of the work, and keeping a record your advisor can follow, and that you can still follow yourself at the end of the term.

**Use whichever agent and model you prefer.** Claude Code, Cursor, Aider and the rest work the same way, and if you already have one set up, keep it. This guide uses **Codex**, OpenAI's *coding agent*, as the common point of reference, because the university's ChatGPT license gives every student in the course access to it at no cost. Where a command or a menu is named below, it is Codex's; the workflow is the same in the others.

Two things:

1. **Using the agent.** How to give it a task it can finish, how to check what it produced, and what to do when it is wrong.
2. **Managing the project.** Issues, milestones, branches and the *wiki* on GitHub, so the history of the project shows what was done and why, not just a list of commits.

**The second is the one you are examined on.** It records which change was made when, why, and what was checked before it was accepted. The first produces code; the second is how you show you understand it.

## Working with an agent, and understanding the result

**Using AI is assumed.** Nothing you hand in is judged on how much came from an agent. What is examined is whether you drove it: the direction was yours, and you adjusted what came back until it met the need you had.

Adjust as you go, not at the end. State what you want, read what arrives, say why it is not right yet — narrower scope, different structure, a case it missed, a name that does not match the rest of the project — and ask again. The first output is a draft. Treating it as the

answer shows up in the presentations, because you cannot explain a decision you did not make.

**Have the agent produce as much of the work as it can, and understand all of it.** The agent writes more code in an afternoon than you would in a week. Whether that code is worth keeping depends on someone being able to say what each part does and what breaks if it changes. Code nobody understands cannot be safely changed later or defended when someone asks about it.

What this asks of you is knowledge of the whole project, not of any one function. You ask Codex for a function that reads your log files and returns a list of events. It writes one, and it works. It does not tell you that `run_experiment.py` already reads those files a different way, that the two disagree on a file with no lines in it, and that the fix belongs in one place rather than two. Accept it and you now have two parsers to keep in step: the next change to the log format has to be made twice, and the run that used the wrong one produces numbers you cannot explain three weeks later.

You decide the things in the right-hand column. The agent will not raise them:

| The agent does this well             | You decide                                             |
|--------------------------------------|--------------------------------------------------------|
| Writing a function, a test, a script | Which part of the project is responsible for what      |
| Renaming, porting, reformatting      | How those parts call each other                        |
| Finding where something is defined   | Which failures matter, and what to measure             |
| Drafting text from your notes        | Which of five designs that all work is the one to keep |

The presentations and the final ask about the right-hand column.

**The review step and the records are how you keep that understanding.** Reading each *diff*, writing each *commit* message, keeping the Log and AGENT-LOG.md current.

## What does the course expect at the end?

These deliverables, which belong to the project rather than to this lecture:

| Deliverable    | What it has to show                                                                           |
|----------------|-----------------------------------------------------------------------------------------------|
| The repository | Branches per task, a commit per change, and commit messages saying what was checked           |
| Issues         | Each closed by the commit that finished it, and assigned to a milestone                       |
| Milestones     | Dated across the 14-week semester, and moved when the schedule changes                        |
| The wiki       | Home, Log with an entry per week, Design, Results, Reading, Appendix                          |
| sandbox/       | A folder per piece of technology you had to try out, each with a README saying what it showed |
| AGENT - LOG.md | What Codex did, and what you checked before accepting it                                      |
| paper/         | The write-up in LaTeX, with its bibliography                                                  |
| Presentations  | Given through the term, with the repository open                                              |

**The repository is checked for progress at each presentation.** What is looked at is when the work happened: commits spread across the weeks, Log entries dated as they happened, issues closed as milestones came due. A repository where every *commit* falls in the last two weeks shows that. Git records when each *commit* was made, so a history cannot be added afterwards.

**The final is a test on your own deliverables.** Your repository is collected into one file and you answer questions about it. **A change you accepted without reading is a question you will not be able to answer.** The section [The final: questions about your own deliverables](#) lists what is collected and the kinds of question asked.

## What is Codex?

A program that reads your repository, proposes changes, runs commands, and reports what it did. You give it a task in words; it works in a sandboxed copy of your repository and comes back with a *diff* you review.

| Piece              | What it is                                                                                         |
|--------------------|----------------------------------------------------------------------------------------------------|
| <b>The model</b>   | GPT-5.1-Codex or whichever model is current. Given text, it produces text                          |
| <b>The harness</b> | The program around the model: it reads your files, runs your tests, applies edits, makes commits   |
| <b>The sandbox</b> | The environment the harness works in — a copy of your repository, with your dependencies installed |

**Use the VS Code extension.** Install **Codex** from the Extensions view in VS Code, sign in with your university ChatGPT account, and open your project folder. The extension shows each proposed change as a *diff* in the editor, next to the file it changes, which is where you will be doing the review described under [Reviewing what Codex produced](#). Everything in this guide assumes you are working that way.

## What should you give the agent, and what should you keep?

Give it tasks with a stated goal and a way to tell when the goal is met. Keep the ones whose goal is not written down anywhere.

| Give it                                                                                   | Keep for yourself                               |
|-------------------------------------------------------------------------------------------|-------------------------------------------------|
| "Add a test for the empty-input case in <code>parse_log</code> "                          | Deciding what the research question is          |
| "This crashes on files over 2 GB — find out why"                                          | Deciding which experiment answers the question  |
| "Rename <code>proc</code> to preprocess everywhere and update the docstrings"             | Judging whether a result is real or an artifact |
| "Write a script that plots accuracy against training size from <code>results.csv</code> " | Interpreting the plot                           |
| "Port this from <code>argparse</code> to <code>click</code> , keeping the same flags"     | Choosing what belongs in the thesis             |

The division is not by difficulty. It is by who is accountable for the answer being right. You are the author of the research. The agent produces code, and nothing in it establishes whether a conclusion holds.

**Three rules for the term**, each worked through in the steps below:

3. **Read every *diff* before you accept it.** Not skim — read.
4. **Make the agent prove it works.** Ask for a test, run it yourself, and check it fails without the change.
5. **One task, one *branch*.**

## Using Codex for the research, not only the code

Codex reads and writes text, and a research project is mostly text. It is as useful outside the source files as inside them, provided you apply the same rule: you check the output.

| Ask it for                                                                      | Then do this                                                                        |
|---------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| A summary of a paper you have downloaded, or the difference between two methods | Read the paper. The summary tells you where to look, and is not a citation          |
| A first draft of a methods section from your code and your AGENT-LOG.md         | Rewrite it in your own words. It cannot know what you decided or why                |
| A reading list on a topic, with the search terms to find it                     | Find each item yourself. A reference that does not resolve was not read by anything |
| A restatement of your research question in one sentence, from your notes        | Judge whether it is still the question you are answering                            |
| Objections to your own argument, or what a reviewer would ask                   | Answer them, or record why they do not apply                                        |

**Two things it must not produce.** A citation you have not opened, and a number you have not reproduced. A plausible reference to a paper that does not exist is the single most common way this work goes wrong, and it is found by clicking the link.

Keep this work in the repository as well. A notes/ folder of Markdown files, committed as you go, gives you a dated record of what you were thinking, and gives Codex the context to be useful the next time you ask.

## Initial Setup

### GitHub access: a key and a token

Do this once, before anything else. Two pieces: an **SSH key** so git can push without a password, and a token so the agent can work with your issues.

**You need a Unix shell.** Every command here is written for one, so use **Linux**, **macOS**, or **WSL** on Windows — Microsoft's Windows Subsystem for Linux, which gives you an Ubuntu shell alongside Windows. `wsl --install` in PowerShell sets it up. On WSL, see [Using VS Code with WSL](#) in the appendix before you start: VS Code does not connect to WSL on its own, and where you keep your files there matters. Windows `cmd` and PowerShell will not do — different quoting, no `ssh-keygen` in the same place — so commands fail in ways that look like project problems.

**An SSH key.** GitHub identifies you by a key pair — a private half that stays on your machine, a public half you give to GitHub:

```
ssh-keygen -t ed25519 -C "you@dh.csudh.edu"
```

Press Enter at each prompt. Then install **gh**, GitHub's command-line tool (`sudo apt install gh`, or `brew install gh` on macOS), and sign in:

```
gh auth login
```

Choose **GitHub.com**, **SSH**, and let it upload the key you just made. Check it worked:

```
ssh -T git@github.com
```

```
Hi yourname! You've successfully authenticated, but GitHub does not provide shell access.
```

That means the key is in place. Clone with the `git@github.com:` form of the URL from here on and `git push` will not ask you for anything. The same key pushes to your *wiki*, which [The research notebook, in the repository wiki](#) uses.

**A fine-grained token, so the agent can file and update issues.** The key covers git, not GitHub's API — which is what gh and Codex use to open, comment on and close issues. A **fine-grained personal access token** limits that access to one repository.

At **Settings → Developer settings → Personal access tokens → Fine-grained tokens → Generate new token**, set:

| Field             | Set it to                                                               |
|-------------------|-------------------------------------------------------------------------|
| Token name        | codex-<your-project>, so you know what it belongs to when you revoke it |
| Expiration        | The end of the semester. Not "no expiration"                            |
| Resource owner    | Your own account                                                        |
| Repository access | <b>Only select repositories</b> → your project repository               |

Then under **Repository permissions**, grant these and nothing else:

| Permission | Level          | Why                                          |
|------------|----------------|----------------------------------------------|
| Metadata   | Read-only      | Required; GitHub selects it for you          |
| Issues     | Read and write | Open, comment on, label and close issues     |
| Contents   | Read and write | Push branches and read files through the API |

Copy the token — GitHub shows it once — and hand it to gh, which stores it:

```
gh auth login --with-token < token.txt
rm token.txt
```

Check what the token can do:

```
gh issue list --repo you/your-project
gh issue create --repo you/your-project --title "Test issue" --body "Checking token access"
```

If that creates an *issue*, the permissions are right. Close it again.

**Never *commit* a token, and never paste one into a prompt** — anything you type to an agent leaves your machine. `gh` stores it outside your repository. If one does end up in a *commit* or a chat, revoke it on the page you created it; a revoked token is harmless.

**The *wiki* is pushed with the key, not the token.** Fine-grained tokens have no *wiki* permission, so cloning `your-project.wiki.git` over HTTPS with one fails. Use the `git@github.com:` URL, as [The research notebook, in the repository wiki](#) does. Codex edits those pages as ordinary files, and you push them.

## The repository

Create a repository on GitHub for the project. Private is fine. Add your advisor as a collaborator. Leave the *branch* settings alone: you work alone here and *merge* your own branches from the command line, so what keeps `main` clean is the habit of working on a *branch* and merging only after the review described under [Reviewing what Codex produced](#).

Clone it and make the first *commit* yourself, by hand:

```
git clone git@github.com:you/your-project.git
cd your-project
```

Write a `README.md` with three things: what the project is trying to find out, how to run it, and what state it is in. Update the third one as you go — it is the file your advisor reads first and the one you will lift into your write-up.

## Milestones, written before any issue

A **milestone** is a dated group of issues. Make them first: they divide the term into phases with dates, which is what the issues then attach to.



In **Issues** → **Milestones** → **New milestone**, create one per phase of the project, each with a due date. The semester runs **14 weeks**, so four to six milestones spaced about three weeks apart covers it. Set the dates against your own section's calendar:

| Milestone            | Due     | What it means                                                           |
|----------------------|---------|-------------------------------------------------------------------------|
| Literature and scope | Week 3  | You can state the question in one sentence and name the work it follows |
| Working pipeline     | Week 6  | Data in, result out, end to end, however crude                          |
| First measurement    | Week 9  | A number you can defend, with the code that produced it                 |
| Experiments complete | Week 11 | Every run the write-up will cite has been done                          |
| Write-up             | Week 14 | The document, the repository, and the reproduction instructions         |

Codex can create them once you have the list and the dates:

```
Create these milestones in this repository with gh, using these due dates:
Literature and scope 2026-02-13, Working pipeline 2026-03-06, ...
```

Your advisor should be able to look at the *milestone* page in week 7 and see whether the project is on schedule. That is the reason for the dates. Weeks 12 to 14 are the write-up, so treat week 11 as the end of the experimental work rather than the start of the last push.

## Writing an issue

An **issue** is one piece of work. Write it as if you were handing it to another student, because you are effectively handing it to the agent.

A usable *issue* has four parts:

```
Title: Empty log files crash parse_log
```

```
Context
```

```
parse_log.py is called by run_experiment.py on every file in data/raw/.
Three of the files from the January capture are zero bytes.
```

```
What is wrong
```

```
parse_log raises IndexError on line 42 when the file has no lines.
Traceback attached.
```

What "done" looks like

parse\_log returns an empty list for an empty file, run\_experiment skips it with a warning, and there is a test for both.

Out of scope

Whether those files should exist at all - that is issue #14.

Label it and assign it to a *milestone*. Two labels are enough: one for the kind of work (bug, experiment, infrastructure, writing) and one for whether it blocks something else.

**Codex can file it for you.** The token from [GitHub access: a key and a token](#) covers issues, so hand it the text:

Open an issue with gh titled "Empty log files crash parse\_log", with this body: <the four sections above>. Put it in the Working pipeline milestone, label it bug.

Read back what it filed, with `gh issue view 23`. You write the four sections; the agent types them into GitHub. An *issue* you did not write is one you cannot answer questions about.

**Why this format.** "What done looks like" is the part the agent needs and the part you will otherwise leave out. Without it, the agent picks its own finish line, and you spend the review arguing about scope instead of correctness. With it, you can hand the *issue* text to Codex almost unchanged.

## The cycle, repeated for every issue

**Codex runs the git commands.** Branching, committing, pushing, merging, tagging: you say what you want and it does them. The commands are printed through these steps so you can see what happened and check it afterwards on GitHub, and the reference at the back says what each one means. What stays with you is the review and the decision to *merge*.

## One issue, one branch

Ask for the *branch*:

Start a branch called 23-empty-log-files from an up-to-date main.

Name branches after the *issue* number: in six weeks you will not remember what fix-parsing-2 was, and 23-empty-log-files matches the *issue* in GitHub's interface.

Then give it the *issue*, not a summary of the *issue*. Paste the text. The context you wrote for a human reader is the context the agent needs.

**Point it at the test, not the symptom.** "Make `test_empty_file` pass" is a task with a finish line. "Fix the parsing bug" is not. If no test exists yet, ask for the test first, confirm it fails, then ask for the fix.

## Reviewing what Codex produced

Four things, in order. They take about five minutes.

**Read the *diff* in the editor.** The extension shows each change as it makes it, in the file it changes, with the old and new lines side by side. Read them there rather than after the fact. A change too long to read through means the *issue* was too big.

**Run the tests yourself.** Do not take the agent's word for it:

```
python -m pytest -q
```

**Check the test is real.** Ask for the fix to be set aside, the tests run, and the fix put back. The new test must fail while the fix is out.

A test that passes with and without the change tests nothing.

**Check what else changed, on GitHub.** Agents reformat files, bump dependencies and delete code they take to be unused. After you push the *branch*, open

```
https://github.com/you/your-project/compare/main...23-empty-log-files
```

which lists every file the *branch* touches and shows each one's *diff*, coloured, with the surrounding lines. Anything in that list you cannot explain, ask about or *revert* before you *merge*.

## Committing every change, then merging

**One *commit* per change the agent makes.** *Commit* as soon as you have reviewed and accepted a change — not four at once, and not at the end of the afternoon. A *branch* built this way can be unwound one change at a time. The *branch*'s history, on GitHub or in the editor, reads:

```
a1c4e02 Skip zero-byte files in run_experiment
7d3b118 Return an empty list for empty log files
2f9e845 Add test_empty_file
```

If the middle one is wrong, reverting it leaves the other two in place. If all three were one *commit*, you would separate them by hand.

**Tell Codex to do the committing.** Put a standing instruction in `AGENTS.md` at the top of your repository. Codex reads it at the start of every session, so you write it once:

```
# Instructions for coding agents

- Work only on the current branch. Never commit to main.
- After each change that leaves the tests passing, make a commit.
- One commit per change. Do not batch unrelated edits.
- Write the commit message yourself: what changed, then why, in the imperative.
- Ask me for the Checked: lines before committing. Do not invent them.
- Do not merge until I say the branch is reviewed and finished.
- When I do, merge into main with --no-ff, push, and delete the branch.
```

*Commit* that file too: its history shows how your instructions changed over the term, which the methods section can cite.

**Review before the *commit*, not after.** The agent commits and merges, but only on a *branch* and only when you say the *branch* is finished, so nothing reaches `main` that you have not read.

**Codex writes the message.** It has the *diff*, so it can summarize what changed and why — which is most of a *commit* message:

```
Return an empty list for empty log files

parse_log indexed lines[0] without checking, so the three zero-byte files
from the January capture crashed run_experiment. Returns [] instead, and
run_experiment skips those files with a warning.

Closes #23
```

`Closes #23` makes GitHub close that *issue* when the *commit* reaches `main`, so the *issue* list stays true without you maintaining it. Ask for it by number when the *commit* finishes an *issue*.

**You add what you checked.** The agent can describe the change; it cannot report a check it did not watch you do. Dictate those lines, or write them yourself before the *commit*:

```
Skip zero-byte files in run_experiment

Codex wrote the fix and the test from the issue text.
```

Checked:

- the test fails with the fix reverted
- run\_experiment now processes all 47 January files, was 44
- no other files touched (git diff --stat: 2 files)

Closes #23

That is the record that you reviewed the work, and what tells you later why you trusted it. Then ask it to push, so the work is not only on your laptop.

**Ask Codex to *merge* it.** You reviewed the changes and the tests pass, so the work is finished; the *merge* is mechanical and the agent does it:

The branch is reviewed and the tests pass. Merge it into main with --no-ff, push main, and delete the branch here and on GitHub.

--no-ff forces a *merge commit* even where git could have moved the *branch* pointer instead. That *commit* records that these three commits were one task and names the *branch*, so main reads as a list of finished tasks with the steps underneath. Deleting the *branch* removes only the name; the commits are in main.

**The agent merges; you decide when.** The AGENTS.md line says it may not *merge* until you say the *branch* is finished, so a *branch* reaches main only after you have read the changes. If you skip the review and say *merge* anyway, unreviewed code is in main under your name.

Then ask for the next *branch* and start again.

That is the cycle, and it repeats for every *issue*:

|                                             |                                                                             |
|---------------------------------------------|-----------------------------------------------------------------------------|
|                                             |                                                                             |
| <b>1. Branch</b>                            | git checkout -b 23-empty-log-files, named after the issue                   |
| <b>2. Work</b>                              | Give Codex the issue text; it edits files in that branch                    |
| <b>3. Review</b>                            | Read the diff in the editor, run the tests, revert-and-check                |
| <b>4. Commit each change</b>                | One commit per change, by you or by Codex under AGENTS.md                   |
| <b>5. Push</b>                              | git push -u origin 23-empty-log-files, ending the last message Closes #23   |
| <b>6. Compare, then tell Codex to merge</b> | The compare/main...<branch> page on GitHub, then say the branch is finished |
| <b>7. Delete the branch</b>                 | git branch -d <branch> and git push origin --delete <branch>                |
| <b>8. Next task</b>                         | git checkout -b 28-plot-accuracy                                            |

**One task per *branch*.** A *branch* holding one *issue* can be abandoned, reverted or merged as a unit; undoing one bad change out of four means picking apart a *merge* you no longer remember. If a *branch* is longer than you can read in one `git diff main`, the *issue* was too large.

**When `main` has moved on.** If you merged another *branch* while this one was open, ask for `main` to be merged into your *branch* before merging back.

If the same lines changed on both sides, `git` stops and marks the conflict in the file. Resolve it yourself: choosing which change to keep is a decision about your project.

## Recording what the agent did

Your advisor, and anyone reading the thesis, needs to know which parts you can defend from first principles and which you accepted from a tool.

Keep a file, `AGENT-LOG.md`, and add a line whenever the agent did something substantial:

```
| Date | Issue | What Codex did | What I checked |
| 2026-02-14 | #23 | Wrote the empty-file guard and its test | Test fails without
the fix; 47/47 files parse |
| 2026-02-19 | #28 | Ported the plotting script from matplotlib to plotly | Compared
both outputs on results.csv; identical to 3 s.f. |
```

Two sentences in your methods section then cover it: which tool, for what kind of work, and how the output was verified. That is the same standard you would apply to a library you did not write.

**What the agent must not do:** produce a result you then report. If Codex runs an experiment and prints a number, you run it again yourself before that number goes in the write-up. Code can be reviewed. A result you did not reproduce cannot.

## The research notebook, in the repository wiki

Every GitHub repository has a **wiki**: a second repository attached to the first, holding Markdown pages, with its own history. Turn it on in **Settings → Features → Wikis**, then open the **Wiki** tab and create the first page — GitHub does not create the *wiki* repository until one page exists.

Use it for the writing that is about the project rather than part of the program:

| Page     | What goes on it                                                            |
|----------|----------------------------------------------------------------------------|
| Home     | The research question, in one sentence, and links to the other pages       |
| Log      | Dated entries: what you tried this week, what happened, what you decided   |
| Design   | Why the pipeline is built the way it is, including approaches you rejected |
| Results  | Each measurement, the commit or tag that produced it, and the conditions   |
| Reading  | Papers, with what each one contributes to your question                    |
| Appendix | Side research: anything you investigated that is not part of the main line |

The *wiki* is a git repository, so you can clone it and edit the pages in VS Code alongside your code, with Codex able to read and draft them:

```
git clone git@github.com:you/your-project.wiki.git
cd your-project.wiki
```

Edit, then `git add -A`, `git commit` and `git push` as usual. Pages are files: `Home.md`, `Log.md`. Link between them with `[[Design]]`.

**The Appendix page is for side research.** Independent projects generate questions off to one side of the main one: how a protocol behaves, why a library returns what it returns, whether a rejected approach would have worked.

Give each one a heading on the Appendix page, or its own page linked from there when it runs long:

```
## Why the API returns 202 and not 200 (2026-03-04)
```

```
The upload endpoint returns 202 Accepted, not 200, so the response body is empty
and the result has to be fetched from the Location header afterwards. This is why
fetch_results retries. Demonstrated in sandbox/http-202/.
```

```
## Considered and rejected: sliding-window averaging (2026-03-11)
```

```
Would have smoothed the spikes in Figure 2, but it also removes the two events
the project is about. Not used.
```

Two lines each, dated. Record why you rejected an approach and you can answer when someone asks why you did not use it; otherwise you work it out again.

**Keep a sandbox/ folder for the programs that go with it.** When the side research is about a piece of technology — a protocol, an API, a file format, a library's behavior — write the smallest program that shows it, and keep it:

```
sandbox/
  http-202/
    README.md          one paragraph: what this demonstrates, and what it showed
    upload_once.py      20 lines, no dependencies beyond the project's
  jsonl-streaming/
    README.md
    read_partial.py
```

Rules that keep the folder useful rather than a second project:

- **Nothing in sandbox/ is imported by the pipeline.** It is illustration, not infrastructure. If a *sandbox program* turns into something the project depends on, move it into the source tree and give it tests.
- **One folder per question**, named after the question rather than the technology.
- **A README.md in each**, one paragraph: what it demonstrates, how to run it, and what it showed. That last part is the reason the folder exists.
- **Commit it like anything else** — *branch*, *commit*, *merge* — so the date is recorded.
- **Link it from the Appendix entry**, and link the entry from the program's README.

This suits an agent: "write the smallest script that shows what this endpoint returns when the file is empty" has a stated goal, no dependencies on the rest of your code, and an output you can check by reading it. Run it yourself and confirm it prints what the *wiki* entry claims.

**Why the *wiki* and not the README.** The README says what the project is and how to run it, and stays short. The *wiki* holds the project's history and reasoning, which is what you draw the write-up from. Keeping them apart lets the Log grow to thirty entries without the README becoming unreadable.

**The *wiki* is a separate repository, so it has separate history.** A `git pull` in your code checkout does not update the *wiki*, and a *commit* there does not appear in your code repository's history. Clone both.



## Writing the paper in LaTeX

The write-up is a LaTeX document, kept in the repository next to the code it describes. Installing a TeX distribution and getting a first document to build is a task to hand to the agent: the goal is stated, and you can tell whether it worked.

The format — document class, citation packages and a page budget — is in the appendix section named [Paper format: sections and page budget](#). Read it before you ask for the skeleton, since the prompt below builds what it describes.

Ask Codex, in your project checkout:

```
Set up LaTeX in this repository so I can build a paper.
```

- Install a TeX distribution, biber and latexmk on this machine (Ubuntu/WSL).
- Create paper/paper.tex: report class, llpt, letterpaper, title page, abstract, and chapters for Introduction, Background and Related Work, Method, Results, Discussion, Conclusion.
- Use biblatex with backend=biber and style=numeric-comp, bibliography in paper/references.bib. Load hyperref, booktabs, graphicx and listings.
- Add a Makefile target `make paper` that builds paper/paper.pdf with latexmk.
- Add paper/\*.aux and the other build products to .gitignore.
- Tell me the commands you ran, and stop before committing.

On Ubuntu or WSL that comes to roughly:

```
sudo apt install texlive-latex-recommended texlive-latex-extra texlive-fonts-
recommended \
                texlive-bibtex-extra texlive-science biber latexmk
```

which is about 1 GB. Then make paper should produce paper/paper.pdf. Build it yourself before accepting the change: a Makefile that works on the agent's copy and not on yours is the common failure.

**What to keep in the repository:** paper/paper.tex, paper/references.bib, the figures, and the Makefile. Build products — .aux, .log, .out, .bbl — go in .gitignore. Whether you *commit* the built PDF is your choice; if you do, *tag* the *commit* you submitted from: ask for a *tag* such as submitted-draft-1.

**Then use the agent on the document the same way you use it on the code:**

| Ask for                                       | Then                                             |
|-----------------------------------------------|--------------------------------------------------|
| A table from results.csv in LaTeX             | Check every number against the file              |
| A figure environment for a plot you generated | Build it and look at the page                    |
| The .bib entry for a paper you have open      | Compare it against the paper's own citation page |
| Tightening a paragraph you wrote              | Read it — you are the author of the argument     |

A .bib entry produced from a title alone can name a real journal, a real year and an author who never wrote it. Check every entry against the paper you read.

## Once a week

A project falls behind gradually, and the repository shows it before you notice it. Once a week, in about twenty minutes:

6. **Close what is done.** Any *issue* whose work is merged.
7. **Look at the *milestone*.** Is it going to be met? If not, move issues out of it now rather than in the last week.
8. **Write at least one *issue* for next week.** Full format. One is the floor, not the target: with an agent finishing a task in an afternoon, several issues are usually open and moving in the same week. If you cannot write "what done looks like", the task is not understood yet, and that is the real finding.
9. **Write up any side research.** An Appendix entry for what you looked into, and a sandbox/ folder if it involved a piece of technology you had to try out.
10. **Write the week's Log entry in the *wiki*.** Three or four sentences: what you tried, what happened, what you decided.
11. **Tag anything worth returning to.** When a run produces a number you might cite, ask for a *tag* — a name like v0.3-first-accuracy fixed to that *commit*, pushed to GitHub.

A *tag* is a name for a *commit*. Six weeks later, "the version that gave 0.71" is a name you have rather than a *commit* you are hunting for.

12. **Update the README's status line.** One sentence.

Bring the *milestone* page to your advisor meeting. It answers "how is it going" with evidence.

## The final: questions about your own deliverables

At the end of the term you take a test on the project you built. It is not a general exam on coding agents or git: the questions are about **your** repository. What gets collected into the file you are examined against:

| From                        | What is in it                                          |
|-----------------------------|--------------------------------------------------------|
| The source tree             | Every file, including <code>sandbox/</code>            |
| <code>git log</code>        | Every commit, with its message and what it touched     |
| Issues                      | Each one, with its comments and which commit closed it |
| Milestones                  | The dates, and what was in each                        |
| The wiki                    | Home, Log, Design, Results, Reading, Appendix          |
| <code>AGENT - LOG.md</code> | What the agent did, and what you checked               |
| <code>paper/</code>         | The project write-up in full, and its bibliography     |

**Everyone gets the same exam.** The questions are written once, and they are not taken from anyone's repository. Each question names a kind of thing every project has and asks you to pick one and explain it. The questions are the same for everyone; your answers come from your own work, and they are checked against your collected file.

The kinds of question to expect:

- Choose a *commit* from your history that fixed a defect. What does the change do, and what breaks without it?
- Take an *issue* you closed. What was the check that made you accept the change?
- Your Appendix records an approach you considered and rejected. Which one, why, and what would have to be true for it to work?
- Name a function in your repository that the agent wrote. Explain what it does, line by line.
- Take a number from your Results page. How was it produced, and how would someone else reproduce it?
- Choose a sentence from your Method section. Point to the code that does what it says.
- Name a reference from your bibliography. What does it contribute to your question, and where in the work did you use it?
- Take a figure from your Results section. Which script drew it, from which data?

You pick the example, so any part of the project could be the one you explain. All of these can be answered by someone who read each *diff*, wrote each *commit* message, kept the *wiki* current, and wrote the paper from that record.

The paper is examined like the code: a sentence in your Method section that no *commit* supports, or a citation you cannot place in the work, is a question you will be asked.

**Have Codex pack the file, and study from it.** Nothing about it is secret, and the agent has every tool the job needs:

Pack this whole project into one Markdown file called COLLECTED.md, for revision. Include, each under its own heading:

- every source file, in a fenced block labelled with its path
- the full commit history with the files each commit touched
- every issue, open and closed, with its body, comments and milestone
- the milestones and their due dates
- every wiki page
- AGENT-LOG.md and paper/paper.tex

Put a table of contents at the top. Skip .venv, .git and anything gitignored.

Each part comes from a different place, which is why one command would not do it:

| Part                         | How the agent gets it                                                                                                 |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Source files                 | Walks the directory tree and reads each file                                                                          |
| Commit history               | git log, with the file list for each commit                                                                           |
| Issues, comments, milestones | gh, using the token from <a href="#">GitHub access: a key and a token</a> — they live on GitHub, not in your checkout |
| Wiki pages                   | Clones the wiki repository, which is separate from the code one                                                       |
| The paper                    | Reads paper/paper.tex from the checkout                                                                               |

Skipping .venv matters: it holds thousands of installed files that are not your work, and including them buries the project in the output.

Read the paper against the repository, not on its own. Every claim in it should reach a *commit*, a Results entry or a sandbox/ program, and the questions are asked in that direction.

If you cannot answer a question from those files in November, go back and read the change.

## What each version-control action does

You ask for these in words and Codex runs them. What matters is the effect, which is what the final asks about.

| Action                 | What it does to your project                                                                                                                    |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Clone</b>           | Copies the repository from GitHub to your machine. Done once                                                                                    |
| <b>Branch</b>          | Starts a separate line of work. Changes on it do not affect <code>main</code> until merged                                                      |
| <b>Commit</b>          | Records one change, with a message, at a point in time. The unit that can be undone                                                             |
| <b>Push</b>            | Sends your commits to GitHub. Until then the work exists only on your laptop                                                                    |
| <b>Pull</b>            | Brings in commits made on GitHub since you last looked                                                                                          |
| <b>Merge</b>           | Brings a branch's commits into <code>main</code> . With <code>--no-ff</code> it also records one commit saying those commits were a single task |
| <b>Delete a branch</b> | Removes the branch name. The commits are in <code>main</code> and are not affected                                                              |
| <b>Revert</b>          | Adds a new commit that undoes an earlier one. Both stay in the history, so the record of what happened is intact                                |
| <b>Stash</b>           | Sets changes aside without committing them, and puts them back later. Used to test with the fix removed                                         |
| <b>Tag</b>             | A permanent name for one commit, so "the version that gave 0.71" is a name rather than something to hunt for                                    |
| <b>Conflict</b>        | Two changes touched the same lines and git cannot choose. You decide which to keep                                                              |

Three things you type yourself, in [GitHub access: a key and a token](#), because they set up the machine rather than the project: `ssh-keygen`, `gh auth login`, and `ssh -T git@github.com`.

## Paper format: sections and page budget

An undergraduate independent research paper of about **40 pages**, single column, 11pt, with numbered sections and a bibliography. Use the report document class: article has no `\chapter`, and at forty pages you want chapters. For citations use **biblatex with the biber backend** and `style=numeric-comp`, which prints [3, 7–9] and handles a `.bib` file

the agent can add entries to. `hyperref` makes the citations and cross-references clickable, `booktabs` gives tables that do not look like spreadsheets, and `listings` sets code.

A page budget that comes to forty. Treat it as the shape to aim at, not a rule:

| Section                        | Pages | What goes in it                                                 |
|--------------------------------|-------|-----------------------------------------------------------------|
| Title page and abstract        | 1     | The question and the result, in 200 words                       |
| 1. Introduction                | 3–4   | The question, why it is open, and what this work contributes    |
| 2. Background and related work | 6–8   | What is already known, with citations, and where your work sits |
| 3. Method                      | 8–10  | What you built and how it works. Figures belong here            |
| 4. Results                     | 8–10  | What you measured, with tables and plots. No interpretation yet |
| 5. Discussion                  | 3–4   | What the results mean, and what limits them                     |
| 6. Conclusion and future work  | 1–2   | What is settled, and what you would do next                     |
| Bibliography                   | 2–3   | 20 to 30 references, all of which you read                      |
| Appendices                     | any   | Long tables, extra derivations, sandbox/ output. Not counted    |

## Running Codex against your own model

The university license comes with a token allowance. If you run it out, Codex can be pointed at a model you run yourself. This is optional. Ollama serves an **OpenAI-compatible** API at <http://localhost:11434/v1>, which is what Codex expects, so the two connect directly.

**On your own machine.** Install Ollama from <https://ollama.com>, then pull a model sized for your hardware:

```
ollama pull qwen2.5-coder:7b      # 4.7 GB, runs on a laptop with 8 GB of RAM
ollama pull gpt-oss:20b          # 13 GB, needs a machine with more memory or a GPU
ollama serve
```

Codex reads `~/.codex/config.toml`. Add the provider and a profile that uses it:

```
[model_providers.ollama]
```

```
name = "Ollama"
base_url = "http://localhost:11434/v1"

[profiles.local]
model = "qwen2.5-coder:7b"
model_provider = "ollama"
```

Select that profile in the extension's model settings. Check

<https://developers.openai.com/codex> first: this part of Codex changes often, and the setting has moved between versions.

**On a cloud server.** A rented GPU instance runs a larger model than a laptop will. Install Ollama on the server the same way, then reach it over an **SSH tunnel** rather than opening the port to the internet:

```
ssh -L 11434:localhost:11434 you@your-server
```

Leave that terminal open. While it is, <http://localhost:11434> on your machine is the server's Ollama, so `config.toml` needs no change. An Ollama port open to the internet is an unauthenticated service anyone can send work to; the tunnel keeps it reachable only by you.

**What to expect.** A 7B or 20B model is far smaller than the one behind Codex. It handles one function, one test, one rename, and loses track of a task spanning several files:

| Use the local model for                       | Use the university allowance for       |
|-----------------------------------------------|----------------------------------------|
| Renames, docstrings, formatting, small tests  | Work that crosses several files        |
| Reading and summarizing code you already have | A bug you cannot locate yourself       |
| Drafting Log and Design pages for the wiki    | Anything you will cite in the write-up |

The review described under [Reviewing what Codex produced](#) matters more here: a smaller model produces wrong code that looks right until you run the test.

## Using VS Code with WSL

WSL and VS Code do not connect on their own. VS Code runs on the Windows side, your project lives on the Linux side, and an extension connects them.

13. Install the **WSL** extension in VS Code (Microsoft's own, in the Extensions view).
14. Open your Ubuntu terminal, `cd` to the project folder, and run `code .` — VS Code opens connected to WSL. The bottom-left corner should read **WSL: Ubuntu**.

15. Use **Terminal** → **New Terminal** inside that window. It is an Ubuntu shell in the project folder, so every command in this guide works there, and the Codex extension runs against that checkout.

Microsoft's guide covers the same ground in more detail:

<https://code.visualstudio.com/docs/remote/wsl>

**Keep your files on the Linux side.** Work in your Ubuntu home directory — `~/` — and not in `/mnt/c/...`, which is your Windows drive seen from Linux. Files there are reachable but slow to read and write, which shows up when the agent reads a whole repository or `git status` runs on every prompt. `~` in the Ubuntu terminal is already the right place, so this only goes wrong if you navigate to `/mnt/c` deliberately.

To find those files from Windows afterwards, type `\\wsl$` in File Explorer, or run `explorer.exe .` from the Ubuntu terminal to open the current folder.

## Glossary

Each term is linked from its first use in the text. To come back afterwards, use your PDF viewer's **Back** or **previous view** command, under its Go or View menu — `Alt+←` in most viewers, `⌘+←` on macOS.

| Term                | What it means                                                                                                              |
|---------------------|----------------------------------------------------------------------------------------------------------------------------|
| <b>Branch</b>       | A named line of commits. Work on one, so <code>main</code> always holds something that works                               |
| <b>Coding agent</b> | A program that calls a model in a loop, acts on the replies by editing files and running commands, and reports what it did |
| <b>Commit</b>       | One recorded change, with a message saying what and why                                                                    |
| <b>Diff</b>         | The lines added and removed between two versions                                                                           |
| <b>Issue</b>        | One piece of work, described on GitHub                                                                                     |
| <b>Merge</b>        | Bringing a branch's commits into another branch                                                                            |
| <b>Milestone</b>    | A dated group of issues                                                                                                    |



|                        |                                                                                                             |
|------------------------|-------------------------------------------------------------------------------------------------------------|
| <b>Revert</b>          | <b>A new commit that undoes an earlier one. The history keeps both</b>                                      |
| <b>Sandbox</b>         | The isolated copy of your repository the agent works in                                                     |
| <b>SSH key</b>         | A pair of files identifying you to GitHub: a private half on your machine, a public half uploaded           |
| <b>Tag</b>             | A permanent name for one commit, such as a version you cited                                                |
| <b>Sandbox program</b> | A small program written to show how one piece of technology behaves, kept apart from the project's own code |
| <b>Wiki</b>            | A second repository attached to yours, holding Markdown pages. Where the research notebook goes             |

## Further reading

- **Codex documentation.** <https://developers.openai.com/codex>
- **GitHub Issues and milestones.** <https://docs.github.com/en/issues>
- **GitHub Issues and project boards** — the planning side, beyond issues and milestones: <https://docs.github.com/en/issues/planning-and-tracking-with-projects>
- **gh manual.** <https://cli.github.com/manual/>
- **Connecting to GitHub with SSH.** <https://docs.github.com/en/authentication/connecting-to-github-with-ssh>
- **Documenting a project with a wiki.** <https://docs.github.com/en/communities/documenting-your-project-with-wikis>
- **latexmk** — what make paper runs: <https://mgeier.github.io/latexmk.html>
- **biblatex and biber** — the citation packages this guide uses, with every option for style= documented: <https://ctan.org/pkg/biblatex>
- **Overleaf's LaTeX documentation**, usable whether or not you use Overleaf: <https://www.overleaf.com/learn>
- **Pro Git**, free online, for anything git this guide does not cover: <https://git-scm.com/book/en/v2>
- **"Best Practices for Using AI When Writing Scientific Manuscripts"** (ACS Nano editorial, 2023) — on disclosure and where responsibility sits: <https://pubs.acs.org/doi/10.1021/acsnano.3c01544>